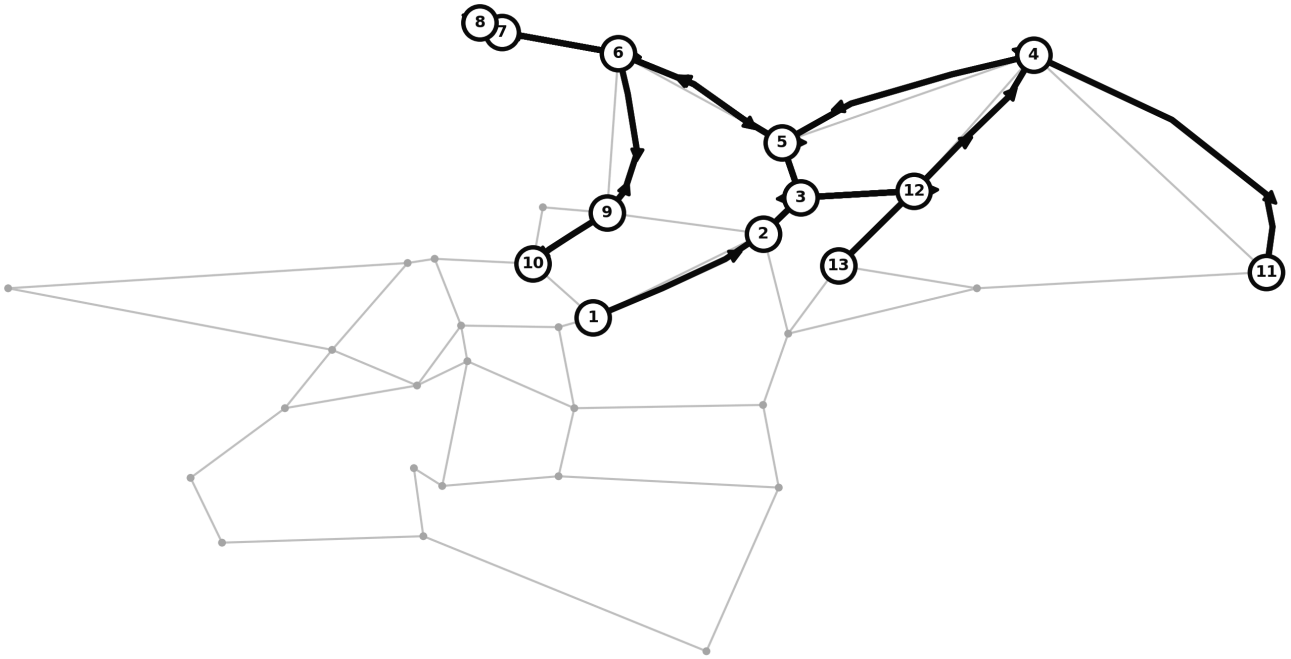
TEAM PROMETHEUS · TRANSPORTATION & LOGISTICS · SOFTWARE

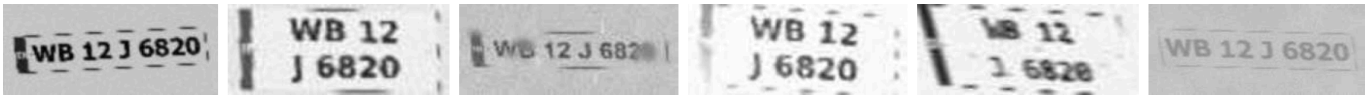
GodsEye

Problem statement — **City-Wide AI Engine for Multi-Camera ANPR Trajectory Tracking and Urban Traffic Analytics**

Every city already owns the cameras. Nothing joins them together. GodsEye reads the number plates those cameras capture and links the readings across the network, so a single vehicle can be followed **across the whole city**.



One vehicle, thirteen cameras, 170 km in a day. The pale web is the 35-camera network; the heavy line is a single registration's real reconstructed journey, in order, along the roads it must have taken. No camera saw more than a fraction of it.



And this is the input. The same number plate as six different cameras actually deliver it — midday, night infra-red, monsoon, headlight glare, high speed, fog.

35
camera sites on a real Kolkata road network

73%
of number plates read correctly

96%
correct among the readings the system stands behind

01 What we have built

Four components, each of which has to work before the next one is worth anything.

(1) **A plate reader for bad photographs.** Not clean studio images — the 9 p.m. capture through rain, at an angle, on a plate half covered in road grime. This is the hard part, and everything else depends on it.

(2) **Vehicle tracking across the city.** Type a plate and get the vehicle's whole day: every camera it passed, in order, drawn on a map, with timings and the road it must have taken between each pair.

(3) **City-wide traffic analytics.** The same readings, aggregated to answer a different question: where the congestion is, which corridor costs the most delay, where traffic enters and leaves.

(4) **A real-time alert queue.** Watch-listed vehicles, duplicated number plates, loitering and unusual routes, raised automatically as readings arrive rather than found by hand.

The design decision everything rests on

The system does not have to answer every time. When it is not confident, it **discards the reading rather than guessing**. So two numbers matter, and they are different: it reads **73%** of plates correctly overall, and of the readings it is confident enough to keep, **96%** are exactly right. A wrong registration put in front of a police officer is far worse than none at all — it accuses the wrong person. The brief asks for above 90%, and section 06 sets out precisely where the shortfall is and what we have already eliminated as its cause.

How it is being tested

We have no access to live city cameras, so we built a simulator: 260 vehicles driving realistic routes across the 35-camera map, with rush-hour demand and standing congestion on the corridors that are genuinely congested in Kolkata.

Every vehicle that passes a camera has a plate image generated and pushed through the *real* reader — errors included. What lands in the database is what the software actually read, not the right answer. The tracking and analytics layers therefore have to cope with imperfect readings exactly as they would in the field.

Swapping the simulator for real cameras means writing rows into a single database table. Nothing above that table knows, or needs to know, where they came from.

ONE SIX-HOUR TEST DAY

Camera crossings	6,586
Readings stored	5,579
Readings refused	1,007
Distinct vehicles	336
Alerts raised	186
Automated self-checks	58 pass

One run of `python seed.py --hours 6`. Counts shift a little between runs: the simulator anchors its window to the clock, and a six-hour window ending at 3 a.m. carries far less traffic than one ending at 6 p.m.

02 How one plate gets read

A camera does not take one photograph of a car — and reading only one is the single biggest thing most systems get wrong.

A traffic camera is triggered as a vehicle enters its zone and takes a **burst** of frames: different distances, exposures and amounts of blur, all of the same plate. Reading only the middle one throws away every other look. The principle is a familiar one — *a single noisy measurement is unreliable; several independent measurements of the same thing are not.*

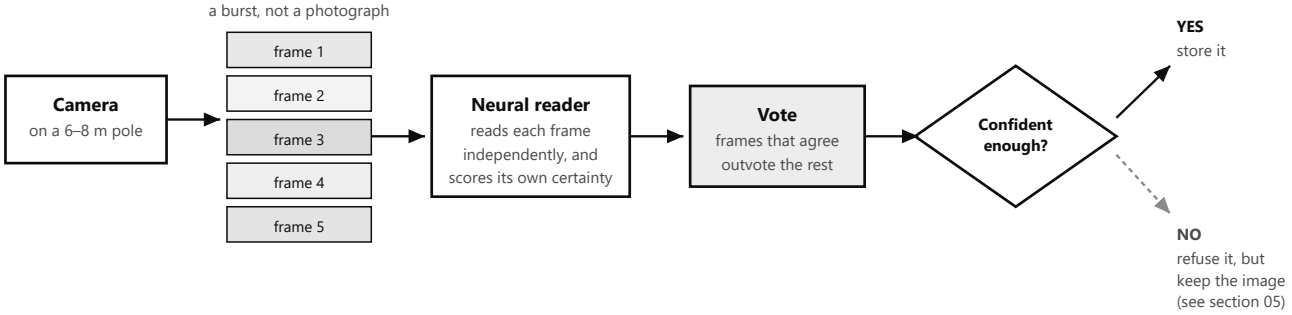


Figure 1 — the reading pipeline. Every stage is measured; none is assumed.

Why the burst matters

Reading a single frame, the software gets **42%** of plates right. Reading five frames of the same vehicle and taking the weighted consensus, it gets **73%**. Nothing about the reader changed — the frames fail in *different* ways, so a frame that loses the plate to glare is outvoted by the four that did not.

Why refusing is a feature

The system applies a confidence threshold and discards anything below it — exactly like reporting "below detection limit" rather than a number you do not trust. It is why 96% of what reaches the database is correct, and why a refusal is a result rather than a failure.

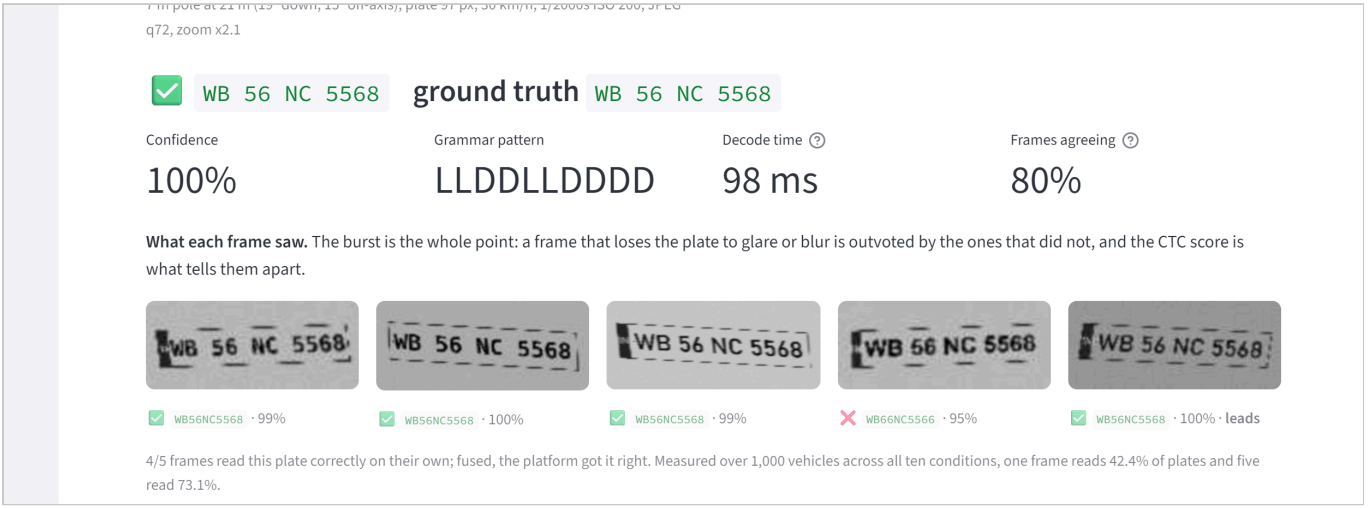


Figure 2 — the reader in burst mode. Five separate captures of one plate, with what each read on its own; the system combines them into a single answer. Here all five agreed.

03 The camera network

The cameras are not a list. They are a network with real roads between them, and that single fact powers everything above the reader.

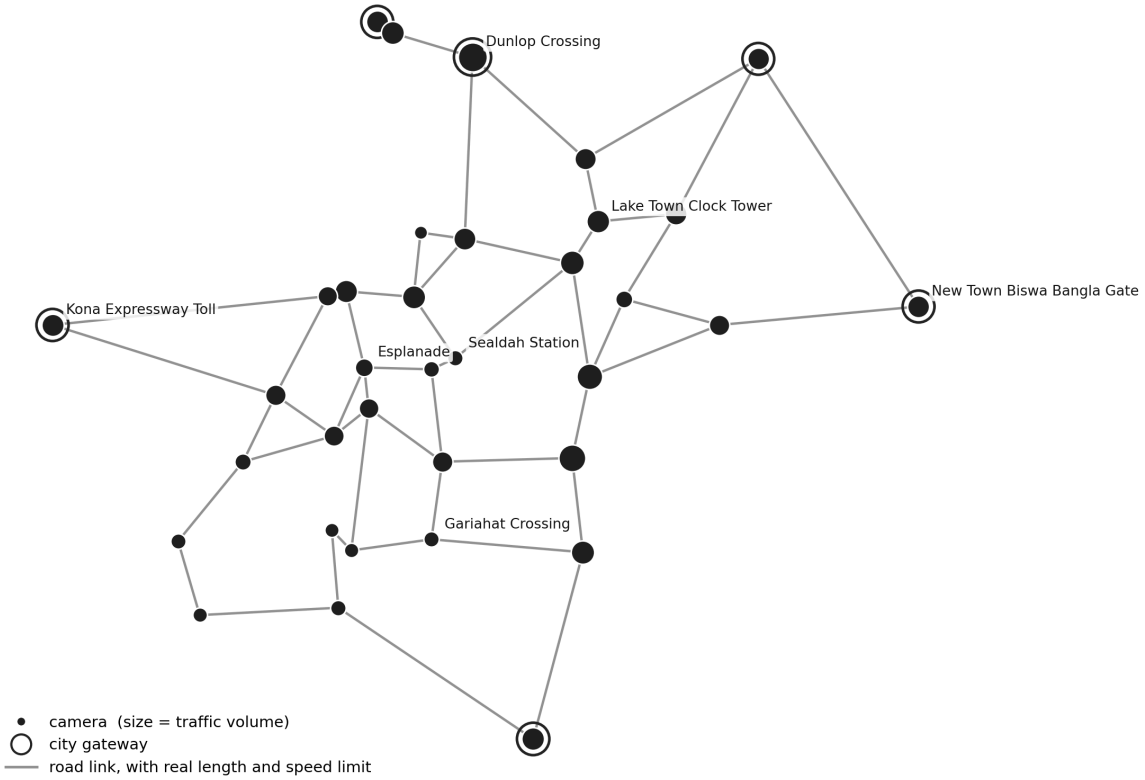


Figure 3 — the network as the software sees it. Each dot is a camera at a real Kolkata junction, sized by the traffic passing it; each line is a road link with a known length and speed limit. Ringed dots are city gateways where traffic enters and leaves.

What the network buys us

Because the software knows the roads, it knows how long a journey between any two cameras *should* take. That one fact powers four separate capabilities:

- drawing a vehicle's route along the road it must actually have taken, rather than a straight line between two dots;
- spotting a journey that is physically impossible — which is how duplicated plates are found (section 05);
- measuring how much delay each corridor is genuinely causing;
- recovering a plate a camera failed to read, by asking which vehicles the neighbouring cameras saw at the right moment (section 05).

Real, and simulated

Real: the reader and the neural network behind it, the plate images including every degradation, journey reconstruction, search, analytics, all six alert rules, and the camera coordinates — 35 genuine Kolkata junctions.

Simulated: the camera feeds, the vehicle movement, and the road lengths and speed limits, which we hand-estimated from maps. In a deployment these would come from the city's own road database.

The honest limit: our plates are generated by our software and photographed through our own model of a camera. Every accuracy figure in this brief measures the reader against that model. It is not a field trial, and we do not present it as one.

04 What an operator does with it

Following one vehicle, seeing the whole city, and being told rather than having to look.

Follow one vehicle

Type a plate number — even a partial or slightly wrong one — and the system rebuilds the vehicle's day. It is deliberately tolerant of its own mistakes: if an operator types `KA05MJ1234` and a rain-blurred camera stored `KA05NJ1234`, the search still finds it, because it knows which character confusions the reader actually makes.

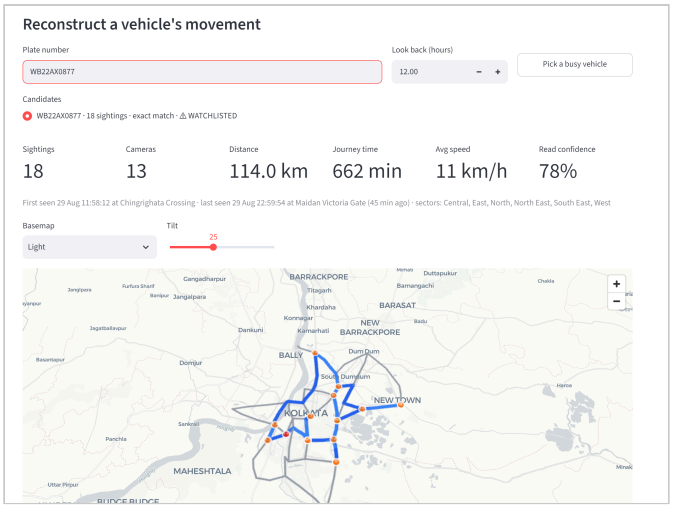


Figure 4 — one vehicle's reconstructed journey: 18 sightings across 13 cameras, 114 km over about eleven hours. The route follows the real roads between cameras, and every leg is listed with its timing and implied speed.

See the whole city

The same readings, aggregated, answer a different question: not *where is this vehicle* but *how is the city moving*. The platform computes traffic density per camera, speed and volume on every road link, a live heatmap, and where journeys begin and end.

One deliberate choice: congestion is ranked by **vehicle-minutes of delay caused**, not by slowness. A slow empty road is not a problem worth an officer's attention. A slow busy one is.

Be told, not have to look

Six rules run automatically as each reading arrives:

Watch-list	a flagged plate is seen anywhere
Duplicate plate	same plate in two places at once
Loitering	same junction 4+ times in 45 min
Detour	returns to a camera the long way round
Speeding	far above the link's free-flow speed
Odd hour	roaming between 1 a.m. and 5 a.m.

Every threshold was set by measurement, not by taste. An early version of the detour rule fired whenever a vehicle covered 1.6 times the direct distance within an hour — which flagged 28% of all vehicles, because that is simply what a commute looks like.

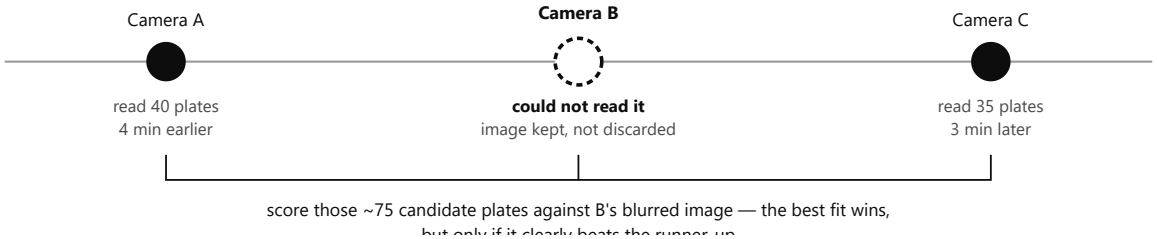
05 Two things we built ourselves

Neither is a library we installed. Both came out of measuring where the system was losing accuracy, and both use the camera network rather than the image alone.

1 · Recovering the readings a camera failed

Using spatial and temporal context to constrain recognition is an established idea; what is ours is the implementation, and the acceptance rules that stop it inventing answers.

When a camera cannot read a plate confidently the reading is thrown away — correct at the time. But minutes later the *neighbouring* cameras have reported, and whatever drove past the failed camera almost certainly appears in one of those lists. So rather than guessing from the 13.6 trillion registrations the plate grammar admits, the system scores the few hundred plates the neighbours actually saw against the original blurred image and asks which one fits best.



Result on the same six-hour day. The system refused 1,007 readings. Only the 25 it had actually put through the reader keep their evidence — the rest were never decoded, so there is nothing to re-score. Of those 25 it recovered **14**, and all 14 were correct. Eleven were genuine repairs of a wrong reading rather than near-misses: `DL3HMV8199` became `OD39HMV8199`, a different state and four different characters.

Safeguard. A recovered plate is stored in a separate table and labelled *inferred*, never mixed with plates that were actually read. An inference is a hypothesis with evidence attached, not an observation — and a guess must never become the evidence for another guess.

2 · Catching duplicated number plates

A cloned plate — the same registration on two vehicles — shows up as a physically impossible journey. The system checks for this **automatically the moment an operator searches**, rather than waiting for somebody to notice.

The trap we had to design around

Our reader is right about 73% of the time, which means it sometimes misreads vehicle X's plate as vehicle Y's — and two different vehicles wrongly sharing one plate number looks *exactly* like a clone. Flagging every impossible journey would turn our own reading errors into accusations against innocent motorists. So before the system will call something a clone it must rule out three cheaper explanations: that one reading was a misread of a similar plate genuinely nearby; that either reading was low-confidence; or that the two cameras are close enough for a clock error to explain it. Only what survives all three is reported, and a single piece of evidence is labelled *suspected* rather than confirmed.

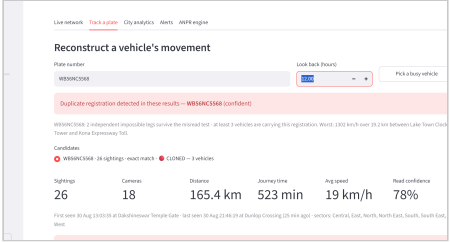


Figure 5 — searching for a plate raised this unprompted, before any result was opened: the registration is on at least three vehicles, and the worst pair implies a journey at 1,302 km/h.

06 Where we stand, and what is next

The brief asks for above 90% plate-reading accuracy. We are at 73%, and we would rather show exactly where the shortfall is than round it up.

CAMERA CONDITION	ONE FRAME	FIVE FRAMES	
Bright daylight	90%	100%	
Night, infra-red lamp	92%	100%	
Heavy rain	62%	98%	
Dusk, poor light	35%	89%	
Oncoming headlight glare	41%	87%	
Fog	32%	83%	
Vehicle moving too fast	41%	80%	
Far side of the road	21%	67%	
Old low-quality camera	10%	27%	
Storm — everything at once	0%	0%	correctly refused
Overall	**42%**	**73%**	

Three problems, not one

- **Storm is not a failure.** We tested whether the plate is recoverable from those pixels by *any* method. It is not. Refusing is the correct behaviour.
- **Seven conditions are fine** — 80% to 100% once the burst is used.
- **Two are genuinely weak** — the far lane and old cameras. We proved the plate *is* present in those images; our reader is not finding it. That is a model problem, and a fixable one.

What we have already ruled out

- **Not too small a model** — a larger one performs the same.
- **Not too little training** — a much longer run came out slightly worse.
- **Not the decoding step** — on 251 failed readings, not one could have been rescued by searching harder.
- **The remaining suspect:** our training images are all heavily degraded, so the reader has learned this camera's flaws rather than the shapes of the letters. It scores 4% on a perfectly clean plate. That is the next experiment.

Next steps

(1) **Real photographs.** The single most valuable thing we could obtain is a few hundred real Indian traffic-camera images. Simulation cannot substitute for them, and we expect our reader to do noticeably worse at first — we would rather know now.

(2) **Fix the training mix.** Our reader has never seen a sharp plate during training. Adding easier examples should teach it letter shapes rather than camera artefacts.

(3) **A trained plate detector.** Given a whole photograph rather than a tight crop, the step that *finds* the plate succeeds only 57% of the time. Replacing it is a well-understood improvement.

(4) **Recognise the vehicle, not just the plate.** A camera sees roughly fifty times more pixels of the car than of its plate, and we discard all of them. Comparing colour and shape across cameras would confirm a recovered plate and settle clone from misread.

Where we would value your guidance. We have deliberately built a system that refuses to answer when unsure, and we have documented our failures alongside our results — including three explanations for the accuracy gap that we tested and disproved. Our question is how to present that honestly to judges without it reading as weakness, when a less careful project can simply claim a higher number.